CASE STUDY · KNOWLEDGE OPS · SEARCH RANKING

Turning a 4,000-page Wiki Into a Ranked Knowledge Asset

A content scoring and search-ranking engine that lifted search success by 25%, cut time-to-find by 30%, and saved 1,200+ analyst hours a year.

+25%	−30%	1,200+ hrs	4,000+
SEARCH SUCCESS RATE	TIME TO FIND ANSWER	ANALYST HOURS / YEAR	PAGES SCORED

Arun · Senior Data Scientist · Mastercard (via Fulcrum Digital)

"I know that doc exists — I just can't find it."

THE WIKI HAD GROWN FAST

Confluence had grown to 4,000+ pages serving thousands of analysts globally. It was the daily lookup tool for docs and playbooks — but searches were returning sprawling, low-relevance results.

WHAT THE LOGS SHOWED

I pulled search logs and saw the pattern: queries that should have had a clear winner were generating multiple clicks and backtracks. The right page existed — it just wasn't ranking.

Search wasn't surfacing the best content

Stale and high-quality pages competed equally — newer, well-maintained docs got buried.

Stale pages crowded the index

Years of accumulated content meant relevance signals were drowning in noise from outdated material.

Real time tax on the org

Multiply small per-search delays across thousands of analysts and you get hundreds of hours a week, lost.

Make search smarter, not just cleaner.

THE PROPOSAL TO MY MANAGER

Don't just clean the wiki — score every page for value, then teach search to rank by that score. Quality content rises, dead weight sinks.

OBJECTIVE

Boost the chance that the right page appears in the top results for any given query.

Search-success oriented

UNIT OF SCORING

Every Confluence page gets a 1–100 composite value score plus a High / Medium / Low tier.

Page-level, transparent

PRIMARY METRICS

Search-success rate (useful click in top results) + median time from first search to landing page.

User-experience grounded

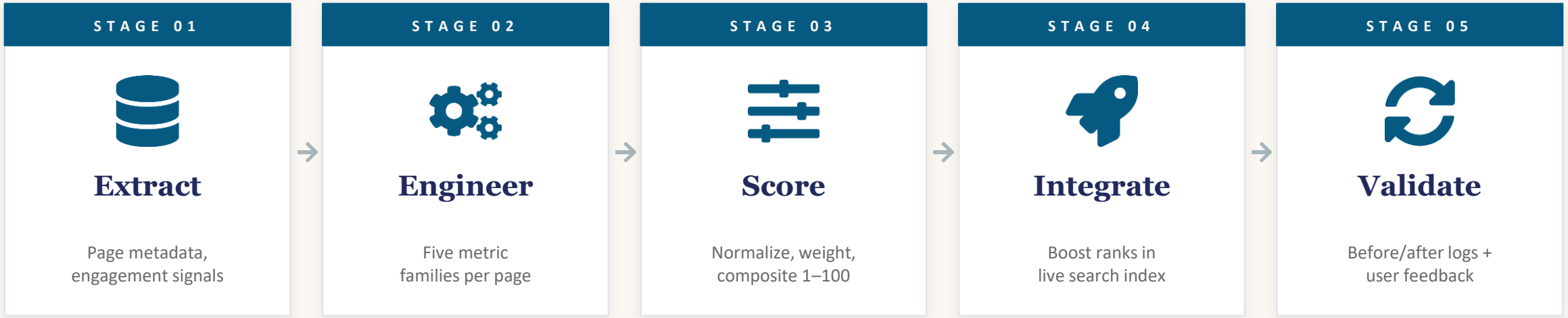
STAKEHOLDERS

Internal Tools team for search integration; power users for weight calibration; content owners for adoption.

Cross-functional from day one

Five stages — from raw page metadata to a smarter search ranker.

I owned the lifecycle: pulling data via the Confluence REST API, engineering the score, working with Internal Tools to integrate it into search, and validating impact.



THE INSIGHT BEHIND THE APPROACH

Most KB problems are framed as content problems. This one was a ranking problem — and the data to fix it was already in the system.

Six signal types pulled per page via the Confluence REST API.

Every page on Confluence emits a continuous stream of behavioral and structural signals. I tapped the REST API to pull all of them, per page, into a unified dataset.

Views

Total page views — the most direct signal of usefulness.

Likes

Active endorsement of content quality by readers.

Comments

Discussion volume — a proxy for actively-used content.

Edit history

Update timestamps and frequency — captures freshness.

Contributors

Distinct authors — pages with one ghost-owner are riskier.

Page hierarchy

Parent / child structure — depth and breadth of coverage.

SCALE · 4,000+ pages × six signal types pulled programmatically — fully automated and re-runnable on cadence.

Five metric families that capture what "valuable" actually means.

Raw signals don't equal value. I rolled them up into five families — each a hypothesis about what makes content useful and trustworthy.

Engagement

Views + likes + comments — captures who's actually reading and acting on the page.

Freshness

Recency of last update — newer maintenance signals living, trustworthy content.

Update cadence

Unique update days + average time between updates — distinguishes a one-off touch from real stewardship.

Content depth & structure

Length and number of child pages — proxies for substantive, well-organized coverage.

Collaboration

Distinct contributors — content with multiple maintainers is more resilient and trustworthy.

WHY FIVE FAMILIES · *A page that scores high on engagement alone could be popular but stale. We needed multiple lenses to define "good."*

Different scales, different importances — handled deliberately.

NORMALIZATION

1 Log transforms where needed

Views and contributors are heavy-tailed — log tames the distribution before z-scoring.

2 Z-score against the corpus

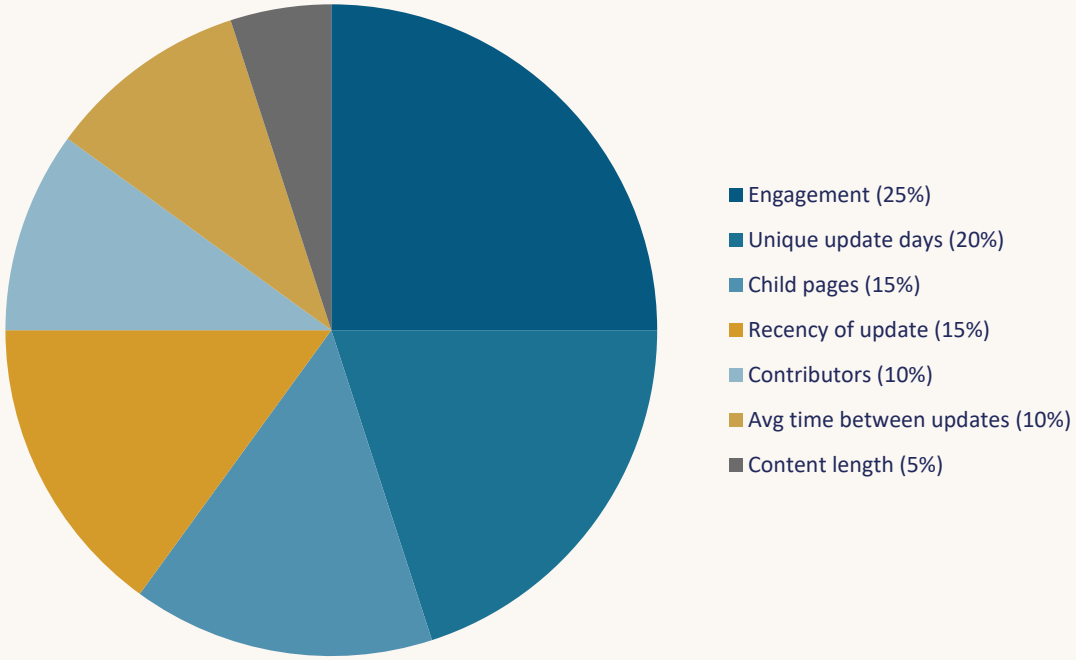
Each metric is standardized against the site-wide mean and std, so they're comparable on the same scale.

3 Weighted composite, scaled 1–100

Final z-scores are weighted, summed, and rescaled to a 1–100 score that's intuitive for stakeholders.

WEIGHTING

Weights set with power users — calibrated to how each signal correlates with "useful, living content."



From a 1–100 score to actionable tiers and flags.

THE COMPOSITE SCORE

FORMULA

$$\text{score(page)} = \Sigma_i \; w_i \cdot z(\text{metric}_i)$$

rescaled to 1–100 across the corpus

TIERING vs. SITE AVERAGE

HIGH	Above ~+0.5 σ — surface in search & recommendations
MEDIUM	Around the mean — standard ranking
LOW	Below ~−0.5 σ — candidate for review or archive

ADDITIONAL FLAGS

Cleanup candidates

Pages in bottom 25% on both views and unique update days — flagged for review or archival.

Unknown owner

Pages without a clear maintainer flagged as quality risks — surfaces orphaned content.

Fresh content

Pages created or updated in the last 90 days highlighted so new material gets discovery.

From scoring table to live search ranking — where users feel it.

Score table

Page-level scores refreshed on cadence

Join to index

Joined to existing Confluence search index

Boost rank

High-score pages lifted in results

Recommend

"Recommended" section for common topics

User searches

Better page in top-3 results

Built ON the existing tool

We didn't replace Confluence search — we boosted it. The score table joined directly to the existing index, so users got a better experience without a new app to learn.

Owner-facing dashboard

Content owners got a dashboard showing their pages' scores and what was dragging them down — turning maintenance into a measurable, visible activity.

INTEGRATION PRINCIPLE · *Don't ask users to change behavior. Make the tool they already use smarter.*

Before-and-after measurement, plus the human signal.

I measured impact on two axes — quantitative log analysis and qualitative power-user feedback — and used both to iterate on weights and thresholds.

QUANTITATIVE — SEARCH LOGS

Top-result click rate

Share of searches where the user clicked a useful page in the top few results — directly captures search success.

Time from first search to landing

Median elapsed time from initial query to the page the user actually settled on — captures real friction.

Backtrack rate

Share of searches with multiple unsuccessful clicks — a strong negative signal of ranking quality.

QUALITATIVE — POWER USERS

Heavy-user feedback sessions

Sat with the analysts who searched most often. Their pain points were sharper than any aggregate metric.

Surfacing edge cases

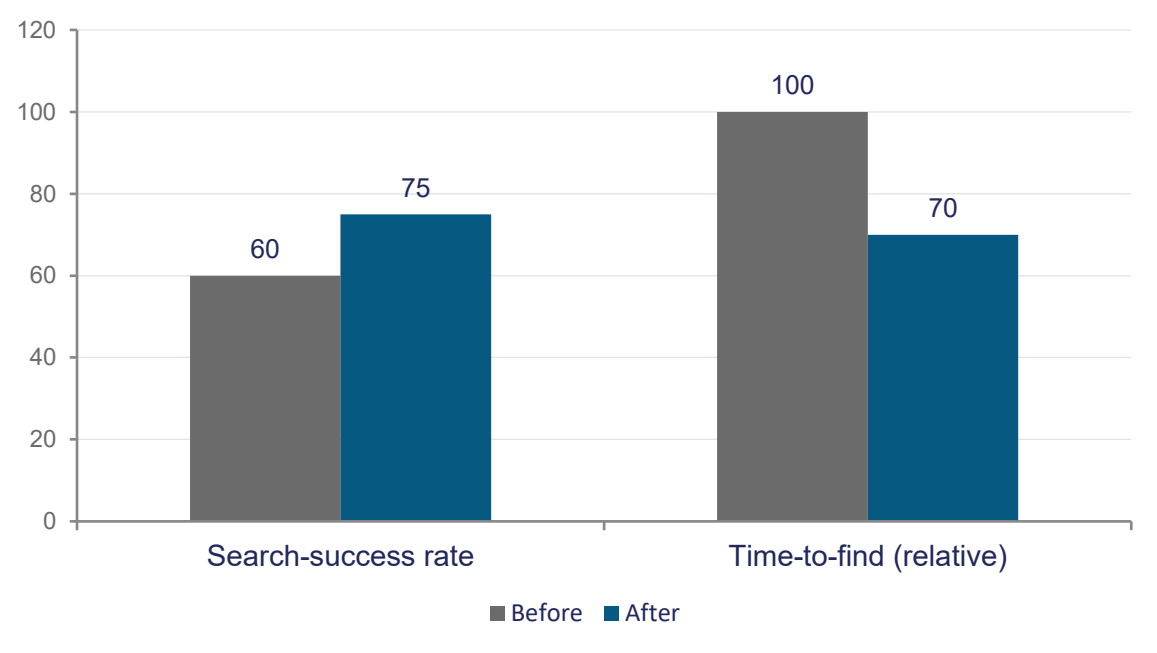
Found query patterns where the score was hurting relevance — fed straight back into weight tuning.

Ongoing iteration

Weights and thresholds tweaked over the first few weeks based on real user response, not just initial intuition.

Search got faster, more accurate, and visibly better-organized.

BEFORE vs. AFTER ROLLOUT



Illustrative — search-success in % terms, time-to-find indexed (Before = 100).

WHAT ALSO CHANGED

Traffic shifted toward quality content

Page views and interactions migrated from stale pages to actively-maintained, high-score ones.

Bottom-quartile decluttered

Hundreds of low-value pages reviewed, archived, or assigned owners — index size meaningfully reduced.

Confluence became a managed asset

Content owners had numbers. "My pages are scoring 45" became actionable rather than abstract.

Reusable scoring pipeline

The pipeline is now a template for clustering, topic modeling, and personalized recommendations.

HEADLINE · +25% search success, −30% time-to-find, ≈1,200+ analyst hours saved per year.

The scoreboard.

SEARCH-SUCCESS RATE

+25%

useful click in top results, before vs. after rollout

TIME-TO-FIND

−30%

median search-to-landing time on common queries

ANALYST HOURS SAVED

1,200+ hrs

estimated yearly time-back across the org

PAGES SCORED & TIERED

4,000+

with cleanup flags, fresh-content highlights, owner risk markers

QUALITATIVE WINS

Traffic shifted toward high-quality content · Confluence reframed as a managed knowledge asset · Pipeline reusable for clustering and personalized recommendations.

What worked, what I'd extend next.

WHAT WORKED

Reframed it as a ranking problem

The org saw a content problem. Rebuilding it as a search-ranking problem unlocked existing data to fix it.

Co-built weights with power users

Calibrating with the analysts who searched most made the score credible — and more accurate from day one.

Boost on existing search, not a new tool

Zero adoption tax. Users got better results in the same UI they already used.

WHAT I'D EXTEND NEXT

Topic modeling for personalization

Cluster pages by topic and personalize ranking by team or role — not all analysts need the same top-3.

Query-side intent signals

Combine page-level scores with query-level intent (recency-seeking vs. canonical-seeking) for sharper ranking.

Automated owner-nudge workflow

Turn the cleanup flags into automatic notifications to content owners, closing the loop without manual review.

13 · TAKEAWAYS

From dumping ground to measurable asset.

01 **Frame the problem one level up.**

The complaint was "I can't find anything." The fix wasn't more cleanup — it was a ranking signal the search index didn't have.

02 **Score what you want to surface.**

A transparent, multi-signal composite score gave us something to rank by, archive against, and have conversations about.

03 **Improve the tool, don't replace it.**

Boosting the existing search beat building a new app. Adoption was zero-effort because the user experience was already familiar.

Thank you · Questions?